```
end
```

The only difference between the for loop and the each form is the scope of local variables that are defined in the body.

You can use for to iterate over any object that responds to the method each, such as an Array or a Range:

```
for i in ['fee', 'fi', 'fo', 'fum']
  print i, " "
end

for i in 1..3
  print i, " "
end

for i in File.open("ordinal").find_all { |line| line =~ /d$/ }
  print i.chomp, " "
end
```

produces:

```
fee fi fo fum 1 2 3 second third
```

As long as your class defines a sensible each method, you can use a for loop to traverse its objects:

```
class Periods
  def each
    yield "Classical"
    yield "Jazz"
    yield "Rock"
  end
end

periods = Periods.new
for genre in periods
  print genre, " "
end
```

produces:

```
Classical Jazz Rock
```

break, redo, and next

The loop control constructs break, redo, and next let you alter the normal flow through a loop or iterator.

break terminates the immediately enclosing loop; control resumes at the statement following the block. redo repeats the current iteration of the loop from the start but without reevaluating the condition or fetching the next element in an iterator. next skips to the end of the loop, effectively starting the next iteration:

```
while (line = gets)
  next  if line.matches?(/^\s*#/)  # skip comments
  break if line.matches?(/^END/)   # stop at end

  # substitute stuff in backticks and try again
  redo if line.gsub!(/`(.*?)`/) { eval($1) }
```